

SOFTWARE DEVELOPMENT - FELLOW

Assignment

Task-Flow - A Personal Task Manager

Full-Stack Mini Project | Estimated Time: 3-4 Hours

1. Overview & Objective

You are asked to build Task-Flow, a simple personal task management web application. This project is intentionally small in scope, designed to be completed in 3-4 hours. The goal is not perfection - it is to observe how you structure a full-stack application, write clean code, and make sensible technical decisions.

You are free to choose any technology stack you are comfortable with. Common choices are listed below as guidance only.

Stack Freedom - Pick What You Know

Frontend: React, Vue, Angular, Svelte, plain HTML/CSS/JS - anything works.

Backend: Node.js / Express, Python / FastAPI or Flask, Go, PHP, Ruby - your choice.

Database: PostgreSQL, MySQL, SQLite, MongoDB - use what you are familiar with.

You will NOT be penalised for choosing a simpler stack. A well-built plain JS + Express + SQLite app beats a broken Next.js + Prisma app every time.

2. What You Will Build

Task-Flow is a browser-based to-do list application where a single user can manage their personal tasks. There is no multi-user or authentication requirement - the focus is on clean CRUD operations and a working full-stack architecture.

#	Feature	Description	Priority
1	View all tasks	Display a list of all tasks with their title, optional description, status, and creation date.	Required
2	Add a task	A form with at minimum a title field. Description is optional. Submitting saves the task to the database via an API call.	Required
3	Mark complete / incomplete	Toggle a task between 'Pending' and 'Completed' states with a single click or checkbox.	Required
4	Delete a task	Remove a task permanently from the database. Prompt the user for confirmation before deletion.	Required
5	Edit a task	Allow the user to update the title or description of an existing task inline or via a modal/form.	Required
6	Filter tasks	Filter the task list by status: All, Pending, or Completed.	Bonus

#	Feature	Description	Priority
7	Input validation	Prevent saving a task with an empty title. Show a clear error message to the user.	Required
8	Persistent storage	All tasks must be stored in a real database (not localStorage or in-memory arrays).	Required

3. API Design Requirements

Your backend must expose a RESTful HTTP API. At minimum, implement the following five endpoints. You may add additional endpoints as needed.

Method	Endpoint	Description	Request Body
GET	/api/tasks	Return all tasks in the database	None
POST	/api/tasks	Create a new task	{ title, description? }
PUT	/api/tasks/:id	Update an existing task (title, description, or status)	{ title?, description?, status? }
PATCH	/api/tasks/:id/toggle	Toggle a task status between pending and completed	None
DELETE	/api/tasks/:id	Delete a task by its ID	None

All API responses should use JSON. Return appropriate HTTP status codes (200, 201, 400, 404, 500). Do not leave all endpoints returning 200 for every scenario.

4. Database Schema

Your database should contain at least one table/collection named tasks (or equivalent). The following schema is a recommended starting point - you may adjust field types to suit your chosen database.

Table: tasks

Column	Type	Notes
id	INTEGER / UUID	Primary key, auto-generated
title	VARCHAR(255)	Required, not null
description	TEXT	Optional, nullable
status	VARCHAR(20)	'pending' 'completed'
created_at	TIMESTAMP	Set automatically on insert
updated_at	TIMESTAMP	Updated on every change

If using a NoSQL database such as MongoDB, apply an equivalent document structure. Ensure created_at and updated_at are present in all documents.

5. Frontend / UI Expectations

The UI does not need to be elaborate or styled with a design system. We are assessing functionality and code structure, not visual design. That said, the interface should be clean, usable, and functional.

Minimum UI Requirements

The following elements must be present in the user interface:

1. A visible list of all tasks showing title, status, and creation date.
2. A form or input area to create a new task (title field at minimum).
3. A toggle or checkbox to mark a task as complete / incomplete.
4. A delete button per task, with a confirmation step.
5. An edit mechanism (inline edit, modal, or separate form).
6. A visible error message when a user tries to submit an empty title.
7. Visual distinction between pending and completed tasks (e.g., strikethrough, colour, icon).

6. Suggested Project Structure

You are not required to follow this exact structure, but your project should be organised and logical. A flat single-file application is not acceptable.

```
task-flow/
├── backend/
│   ├── src/
│   │   ├── routes/           # API route definitions
│   │   ├── controllers/      # Request handlers / business logic
│   │   ├── models/           # Database models or queries
│   │   ├── app.js             # Server entry point
│   │   ├── .env.example       # Example environment variables
│   │   └── package.json
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/       # Reusable UI components
│   │   ├── services/         # API call functions
│   │   ├── App.js             # Root component / entry point
│   │   └── package.json
└── README.md                 # Setup and run instructions
```

If you use a full-stack framework (e.g., Next.js), a monorepo structure is fine. Adjust the layout accordingly and document it in your README.

7. Technical Requirements & Constraints

7.1 Must-Have Requirements

- ✓ All task data must be persisted in a real database - no in-memory arrays or localStorage as the primary store.
- ✓ The frontend must communicate with the backend exclusively via HTTP API calls (fetch / axios or equivalent).
- ✓ Form validation must be implemented on both the frontend (user feedback) and the backend (reject invalid requests with a 400 status).
- ✓ The API must return meaningful HTTP status codes for all operations.
- ✓ The application must run locally by following the instructions in your README.
- ✓ All required features (view, add, edit, complete, delete) must be fully functional.
- ✓ Code must be organised into multiple files - no single-file monolith for the entire app.

7.2 Must-Not-Do (Avoid These)

- X** Do not use a frontend-only database (e.g., IndexedDB or localStorage) as the sole data store.
- X** Do not hardcode task data as JavaScript arrays in your source code.
- X** Do not expose database credentials in your code - use environment variables and provide a .env.example file.
- X** Do not skip the README. An application that cannot be set up from your instructions will not be evaluated.
- X** Do not use a CSS framework as a substitute for understanding basic layout - at least write some of your own CSS.

7.3 Bonus (Optional - Extra Credit)

- ★ Implement a filter bar to toggle between All, Pending, and Completed tasks.
- ★ Add a due date field to tasks and highlight overdue tasks visually.
- ★ Write at least one backend unit test (e.g., for the task creation endpoint).
- ★ Deploy the application (Render, Railway, Vercel, etc.) and include the live URL in your README.
- ★ Add a task priority field (Low / Medium / High) and sort tasks by priority.

8. Submission Instructions

Please follow these submission steps carefully. Incomplete submissions will be reviewed with available materials, but missing items may affect your score.

Step 1: Push your code to a public GitHub (or GitLab) repository.

Use a clear repository name such as task-flow or taskflow-yourname.

Step 2: Include a README.md at the root of the repository.

The README must contain: project description, tech stack used, prerequisites (Node version, Python version, etc.), step-by-step local setup instructions, how to seed or initialise the database, and how to run both frontend and backend.

Step 3: Include a .env.example file.

List all environment variables your application requires (e.g., DATABASE_URL, PORT) with placeholder values. Never commit your real .env file.

Step 4: Share the repository link.

Send the link to your assigned contact before the deadline. Late submissions without prior notice will not be accepted.

Step 5: (Optional) Record a short Loom or screen recording (2-5 minutes).

Walk through the running application and briefly explain one technical decision you made. This is not mandatory but is strongly encouraged - it gives you an opportunity to showcase your thinking.

9. Evaluation Rubric

Your submission will be scored across five dimensions. Each dimension is rated out of 20 points, for a total of 100 points. The pass mark is 60 / 100.

Criterion	Weight	What We Look For
Functionality	20 pts	All required features work end-to-end: create, read, update, delete, toggle status. No broken buttons, blank screens, or 500 errors on normal user flows.
Code Quality & Organisation	20 pts	Code is readable, consistently formatted, and logically split across files. Sensible naming, no dead code, no commented-out blocks left in production paths.
API Design & HTTP Standards	20 pts	RESTful endpoint structure, correct HTTP methods, meaningful status codes, JSON responses with consistent shape. Validation errors return 400, not found returns 404.
Database Integration	20 pts	Data is truly persisted. Schema is appropriate. SQL queries or ORM usage are correct and do not expose injection vulnerabilities. Credentials are not hardcoded.
README & Setup Experience	20 pts	A reviewer can clone the repo and run the app in under 10 minutes by following the README alone. Environment variables are documented. Database setup steps are included.

Scoring Note

Bonus features (filter, due dates, tests, deployment) can earn up to 10 additional points on top of 100. However, bonus work will only be evaluated if all required features are fully functional. A clean, complete app with no bonuses always scores higher than a feature-rich but broken submission.

10. Frequently Asked Questions

Q: Can I use a starter template or boilerplate?

Yes, you may use create-react-app, Vite, or a backend generator to scaffold your project. You must write all application logic yourself - copy-pasting complete feature implementations from tutorials is not acceptable.

Q: Can I use an ORM (e.g., Sequelize, Prisma, SQLAlchemy)?

Yes. Using an ORM is a perfectly valid choice. You will not be penalised or rewarded specifically for using one versus raw SQL - what matters is that it is used correctly.

Q: Does the UI need to be mobile-responsive?

Yes. Design mobile responsive UI.

Q: What if I cannot finish everything in 3-4 hours?

Submit what you have. Leave a note in your README explaining which features are incomplete and what you would have done given more time. Partial credit is available, and your written reasoning is valuable.

Q: Can I use AI coding assistants (e.g., GitHub Copilot)?

Yes, with honesty. If you use AI assistance, briefly mention it in your README. We are evaluating your understanding and judgement, not your typing speed. Be prepared to explain every part of your code in a follow-up technical interview.

Good luck!

We are rooting for you. Focus on getting things working correctly, keep your code clean, and write a README that makes setup effortless.

A working, readable, well-documented project will always impress more than an ambitious but broken one.